

Inheritance: Building New Classes from Old Ones

Quiz 04 constructor chaining, access control, overriding vs. overloading · 15 min · front & back

Name _____

Date _____

array[0] Predict the Output

construction & destruction order

THREE-LEVEL HIERARCHY

```
class Base { public:
    Base() { cout << "B "; }   ~Base() { cout << "~B "; }
};
class Mid : public Base { public:
    Mid() { cout << "M "; }   ~Mid() { cout << "~M "; }
};
class Top : public Mid { public:
    Top() { cout << "T "; }   ~Top() { cout << "~T "; }
};
int main() {
    Top t;
    cout << "| ";
} // t goes out of scope here
```

Write the **exact** output of the whole program, including anything that prints after the `|`.

OUTPUT

array[1] Who Can See What?

private / protected / public

FOUR MARKED LINES

```
class Account {
    double balance;
protected:
    string owner;
public:
    Account(string o, double b) : balance(b), owner(o) {}
    double getBalance() const { return balance; }
};
class Savings : public Account {
    double rate;
public:
    Savings(string o, double b, double r) : Account(o, b), rate(r) {}
    void addInterest() {
        balance += balance * rate;           // Line A
        cout << owner << "\n";              // Line B
    }
};
int main() {
    Savings s("Ana", 100.0, 0.05);
    cout << s.getBalance();                  // Line C
    cout << s.owner;                          // Line D
}
```

For each line, circle whether it compiles and give a short reason.

LINE	COMPILES?	REASON
A	Y / N	
B	Y / N	
C	Y / N	
D	Y / N	

Line A is supposed to work. Without making `balance` `public`, what is the **smallest change** to `Account` that lets it compile?

array[2] Override, Overload, or Neither?

same name ≠ same thing

NO VIRTUAL ANYWHERE

```
class Animal { public:
    void speak()      { cout << "..."; }
    void eat(string food) { cout << "eats " << food; }
};
class Dog : public Animal { public:
    void speak()      { cout << "Woof"; }
    void speak(int times) { for (int i = 0; i < times; i++) cout << "Woof "; }
    void fetch()        { cout << "fetching"; }
};
```

(a) Label each pair **OVERRIDDEN**, **OVERLOADED**, or **NEITHER**.

(b) Given `Animal a;` `Dog d;` what does each call print?

PAIR	ANSWER
<code>Animal::speak()</code> <code>Dog::speak()</code>	
<code>Dog::speak()</code> <code>Dog::speak(int)</code>	
<code>Animal::eat(string)</code> <code>Dog::fetch()</code>	

CALL	PRINTS
<code>a.speak();</code>	
<code>d.speak();</code>	
<code>d.eat("bone");</code>	

(c) Write the **one statement** you would put inside `Dog::speak()` to also run `Animal`'s version.

array[3] Write the Code

derive, chain, override

BASE CLASS — DO NOT CHANGE

```
class Employee {
protected:
    string name; double salary;
public:
    Employee(string n, double s) : name(n), salary(s) {}
    void print() { cout << name << ": $" << salary << "\n"; }
};
```

Write a class `Manager` that **publicly** inherits from `Employee` and has:

- a private `int reports;` (number of direct reports)
- a constructor `Manager(string n, double s, int r)` that passes `n` and `s` up to `Employee` using an **initializer list**
- its own `print()` that first runs `Employee`'s `print()`, then prints `Reports:` followed by `reports`